

Automatic Text Summarization System Using Natural Language Processing

Group 34 Intelligent System Project Work

Submitted by:

Group 34

Ofori Kelvin Antwi	PS/ITC/22/0153
Perpetual Oppong	PS/ITC/22/0086
Benedict Quaicoe	PS/ITC/22/0051
Prince Kofi Amanpeni	PS/ITC/22/0119
Collins Adomako	PS/ITC/22/0060
Nana Kwadwo Safo	PS/ITC/22/0213
Ali Alhassan Abdul Aziz	PS/ITC/22/0121

July 2, 2026

Source code: <https://github.com/KelvinOfori2/SummarizeAi>

Tools used: Python 3.12, FastAPI, Sumy, Scikit-learn, NLTK,
PyMuPDF, React 19, PostgreSQL, Alembic, Tailwind CSS

0. Contents

1	Introduction	2
2	Problem Statement	2
3	Objectives	2
4	Methodology	3
4.1	Extractive Summarization Approach	3
4.2	Algorithm Overview	3
4.3	Text Preprocessing Pipeline	3
5	System Design	4
5.1	Architecture	4
5.2	Backend Module Structure	4
5.3	Database Schema	4
5.4	API Endpoints	5
5.5	Frontend Page Structure	5
6	Implementation	5
6.1	Backend	5
6.2	Frontend	6
6.3	Authentication Flow	6
6.4	Development Setup	6
7	Testing and Results	7
7.1	Functional Testing	7
7.2	Algorithm Performance Comparison	7
7.3	PDF Noise Reduction	7
7.4	Rate Limiting and Security	8
8	Conclusion	8
9	Recommendations	8
	References	9

1. Introduction

Reading every document in full is rarely practical. Academic papers, technical reports, and long-form articles routinely run to dozens of pages when the relevant content might fill two or three. Manual summarization is slow and inconsistent; extractive NLP methods address both problems without requiring training data or cloud services.

This project implements a full-stack **Automatic Text Summarization System** that applies four extractive NLP algorithms to user-supplied documents. Users can paste raw text or upload PDF, DOCX, or TXT files up to 10 MB. The system returns a condensed summary composed entirely of sentences drawn verbatim from the source, along with word-count statistics, compression ratio, and processing time. Summaries can be exported as PDF, DOCX, or plain text.

The system uses classical, unsupervised NLP methods rather than large language models: TF-IDF, Latent Semantic Analysis (LSA), LexRank, and Luhn. Because no new text is generated, every sentence in the output appeared verbatim in the source document.

2. Problem Statement

Professionals, students, and researchers regularly work through large documents (security reports, academic papers, meeting transcripts) where extracting the key content by hand is slow and inconsistent. Existing summarization tools fall short in several ways:

- Generative AI models may introduce fabricated content not present in the source,
- Cloud-based tools require a network connection and often charge per request,
- Most tools give no indication of which sentences were selected or why,
- Few handle structured file formats such as PDF and DOCX without manual conversion.

A useful alternative would let users choose their algorithm and output length, work offline, show which sentences were selected, and accept PDF and DOCX files directly.

3. Objectives

1. Implement four extractive NLP summarization algorithms: TF-IDF, LSA, LexRank, and Luhn.
2. Support plain text input and structured file uploads (PDF, DOCX, TXT up to 10 MB).
3. Clean and normalize extracted PDF text to remove noise (headers, footers, table rows, page numbers) before summarization.
4. Allow users to select their preferred algorithm and adjust the output compression ratio (5%–95%).
5. Expose a RESTful API built with FastAPI for all summarization and user-management operations.
6. Build a responsive React 19 web interface with summary history, statistics, and export functionality.
7. Store summaries per-account in PostgreSQL with Alembic-managed schema migrations.
8. Support export of summaries in PDF, DOCX, and plain text formats.
9. Implement JWT-based authentication and role-based access control (user / admin).

4. Methodology

4.1 Extractive Summarization Approach

Extractive summarization scores each sentence in the source document and returns the top- k sentences in their original order, where k is determined by the user-specified compression ratio. Because no new text is generated, the approach cannot produce fabricated content and returns the same output for the same input every time.

4.2 Algorithm Overview

TF-IDF (Term Frequency–Inverse Document Frequency). Each sentence is represented as a TF-IDF vector using Scikit-learn's `TfidfVectorizer`. Sentence importance is the sum of the TF-IDF weights of its constituent terms. Sentences containing terms that appear often in this document but rarely across documents score highly, which makes TF-IDF a good fit for technical reports with specialized vocabulary.

$$\text{score}(s) = \sum_{t \in s} \text{tf}(t, s) \cdot \log\left(\frac{N}{\text{df}(t)}\right) \quad (1)$$

LSA (Latent Semantic Analysis). A term–sentence matrix is constructed and decomposed via truncated Singular Value Decomposition (SVD). Each sentence's score is its squared norm in the reduced semantic space. LSA can surface sentences that share a topic even when they share no exact words, which pure term-frequency methods cannot do.

LexRank. A cosine-similarity graph is built over all sentences, then sentences are ranked by the stationary distribution of a Markov chain over this graph (analogous to PageRank). Sentences that are similar to many other sentences score highest. This approach works well for news and longer prose where sentence diversity matters.

Luhn. Sentences are scored by the density of significant words within a sliding window. Significant words are those appearing above a low-frequency threshold but below a stop-word frequency ceiling. The method is the fastest of the four and works well on short, focused documents.

4.3 Text Preprocessing Pipeline

Before any algorithm runs, the raw text passes through:

1. **PDF noise removal** (for uploaded PDFs): repeated lines appearing three or more times are identified as headers/footers and removed; standalone digit lines (page numbers), CVSS/vector annotation lines, and lines with less than 25% alphabetic content (table rows, code fragments) are stripped.
2. **Sentence segmentation:** NLTK's Punkt tokenizer splits the cleaned text into sentences.
3. **Length filter:** sentences shorter than 30 characters are discarded to prevent single-word fragments from polluting the scoring.

5. System Design

5.1 Architecture

The system follows a client–server architecture:

- **Frontend:** React 19 SPA (Vite, TanStack Query, Tailwind CSS) served on port 5173.
- **Backend:** FastAPI application served by Uvicorn on port 8000.
- **Database:** PostgreSQL 16, accessed via SQLAlchemy ORM with Alembic migrations.
- **Real-time:** WebSocket endpoint for live notification delivery.

5.2 Backend Module Structure

```
backend/
  app/
    api/v1/          # Route handlers (auth, summaries, users,
                     # admin, ws)
    core/            # Config, security (JWT), sanitization,
                     # logging
    db/              # SQLAlchemy session, base, init_db
    middleware/      # Rate limiter, request logger, security
                     # headers
    models/          # ORM models (User, Summary, ActivityLog,
                     # ...)
    schemas/         # Pydantic request/response schemas
    services/
      file_service.py # PDF/DOCX/TXT extraction and cleaning
      export_service.py # PDF/DOCX/TXT export generation
    nlp/
      preprocessor.py # Sentence tokenization and filtering
      algorithms/     # tfidf.py, lsa.py, lexrank.py, luhn.py
```

Listing 1: Backend directory layout

5.3 Database Schema

Table 1: Core database tables

Table	Key columns	Purpose
users	id, username, email, password_hash, role	Account storage
summaries	id, user_id, algorithm, original_text, summary_text, original_word_count, summary_word_count, compression_ratio	Summarization results
activity_logs	id, user_id, action, created_at	Audit trail
notifications	id, user_id, message, read, created_at	In-app alerts
password_resets	id, user_id, token, expires_at	Password recovery
settings	id, key, value	Admin configuration

5.4 API Endpoints

Table 2: Selected REST API endpoints

Method	Path	Description
POST	/api/v1/auth/login	Authenticate; return JWT
POST	/api/v1/auth/register	Create user account
POST	/api/v1/summaries/	Summarize text
POST	/api/v1/summaries/upload	Upload file and summarize
GET	/api/v1/summaries/	Paginated summary history
GET	/api/v1/summaries/{id}/export	Download in PDF/DOCX/TXT
GET	/api/v1/users/stats	Per-user statistics
GET	/api/v1/admin/users	Admin: list all users
WS	/api/v1/ws/{user_id}	Real-time notifications

5.5 Frontend Page Structure

- **Landing:** public marketing page with algorithm descriptions and feature list.
- **Login / Register:** JWT authentication forms.
- **Dashboard:** stat cards (total summaries, words processed, words saved, last active), recent summaries list, quick actions.
- **Summarize:** text input or file upload, algorithm selector, ratio slider, result display with copy and export.
- **History:** searchable, paginated list of all past summaries with full text view.
- **Profile:** account details and password change.
- **Admin:** overview stats, user management, summary browser, activity logs, settings.

6. Implementation

6.1 Backend

FastAPI was chosen for its async-native design and automatic OpenAPI documentation generation. SQLAlchemy provides the ORM layer; all schema changes are version-controlled with Alembic. Password hashing uses bcrypt via passlib; tokens are signed RS256 JWTs with a configurable expiry.

```

from sumy.summarizers.lex_rank import LexRankSummarizer
from sumy.summarizers.lsa import LsaSummarizer
from sumy.summarizers.luhn import LuhnSummarizer
from sklearn.feature_extraction.text import TfidfVectorizer
import numpy as np

def summarize(text: str, algorithm: str, ratio: float) -> str:
    sentences = preprocess(text)          # tokenise + filter <
    30 chars
    n = max(1, round(len(sentences) * ratio))
    if algorithm == "tfidf":
        return _tfidf(sentences, n)
    elif algorithm == "lsa":

```

```
    return _lsa(sentences, n)
elif algorithm == "lexrank":
    return _lexrank(sentences, n)
elif algorithm == "luhn":
    return _luhn(sentences, n)
```

Listing 2: Core summarization dispatcher (simplified)

PDF files are processed with PyMuPDF (`fitz`); DOCX files with `python-docx`. Before NLP, extracted PDF text is cleaned by the `_clean_pdf_text()` function, which detects and removes repeated header/footer lines, page-number-only lines, and low-alphabetic-content rows (table cells, CVSS vectors).

6.2 Frontend

The React 19 SPA communicates with the backend exclusively through Axios with a base URL configured from environment variables. All server state is managed by TanStack Query v5, which provides caching, background re-fetch, and optimistic updates. Framer Motion provides minimal entrance animations. The design system uses IBM Plex Serif for display headings, IBM Plex Sans for body text, and a warm amber accent colour (`#f59e0b`) on a dark stone background (`#0c0a09`).

6.3 Authentication Flow

1. Client sends `POST /api/v1/auth/login` with email and password.
2. Backend validates credentials, returns a signed JWT (24-hour expiry).
3. Client stores the token in `localStorage` via Zustand's `authStore`.
4. All subsequent requests attach the token as `Authorization: Bearer <token>`.
5. The FastAPI `get_current_user` dependency decodes the token on every protected route.

6.4 Development Setup

```
# Backend
uv venv --python 3.12 && source .venv/bin/activate
uv pip install -r requirements.txt
alembic upgrade head
python -c "from app.db.init_db import init_db; init_db()"
uvicorn app.main:app --reload

# Frontend
pnpm install
pnpm dev
```

Listing 3: Environment setup commands

7. Testing and Results

7.1 Functional Testing

Each major feature was tested manually through the web interface and via the auto-generated FastAPI Swagger UI at /docs:

Table 3: Functional test summary

Feature	Test case	Result
User registration	Valid and duplicate email	Pass
Login / JWT	Correct and incorrect credentials	Pass
Text summarization	All four algorithms, various ratios	Pass
PDF upload	Text PDF and scanned PDF	Pass / Warn
DOCX upload	Standard and malformed DOCX	Pass
Export (PDF)	Download and open in Evince	Pass
Export (DOCX)	Download and open in LibreOffice	Pass
History pagination	20+ summaries, navigate pages	Pass
Admin user listing	View, search, and deactivate users	Pass
WebSocket notify	Login on two tabs simultaneously	Pass

7.2 Algorithm Performance Comparison

A 3,200-word cybersecurity assessment PDF was used as the benchmark document. Each algorithm was run at a 20% compression target.

Table 4: Algorithm comparison on a 3,200-word PDF

Algorithm	Sentences in	Sentences out	Time (s)	Observation
TF-IDF	87	18	0.12	Favoured sentences with domain-specific terminology
LSA	87	18	0.19	Good topical breadth across document sections
LexRank	87	18	0.34	Most coherent reading flow; slowest due to graph build
Luhn	87	18	0.08	Fastest; occasionally picks verbose transitional sentences

7.3 PDF Noise Reduction

Before the `_clean_pdf_text()` function was added, the same PDF produced 312 input sentences, with 67% being fragments shorter than 30 characters (table cells, CVSS rows, page numbers). After cleaning, the sentence count dropped to 87, and summary quality improved considerably. The scoring algorithms no longer competed with repeated boilerplate and single-number rows for ranking positions.

7.4 Rate Limiting and Security

The middleware stack enforces 60 requests/minute per IP using a sliding-window counter in memory. Security headers (CSP, X-Frame-Options, X-Content-Type-Options, HSTS) are injected by a custom `SecurityHeadersMiddleware`. All user-supplied text is sanitized with `bleach` before storage to prevent XSS.

8. Conclusion

This project delivers a complete, deployable text summarization platform built with open-source tools and unsupervised NLP algorithms. The four algorithms (TF-IDF, LSA, LexRank, and Luhn) suit different document types, from Luhn's fast heuristic scoring on short focused text to LexRank's graph-based ranking on longer prose.

The contribution beyond a basic NLP script is the surrounding infrastructure: per-account summary history, multi-format file support, three export formats, a WebSocket notification layer, and an admin panel. The PDF cleaning pipeline addresses a problem most open-source summarization tools ignore: noisy structured documents where table rows, repeated headers, and annotation lines compete with actual sentences during scoring. Cleaning this noise before ranking improved output quality without adding any machine learning.

All project objectives were met:

- Four extractive algorithms implemented and accessible via the UI.
- PDF, DOCX, and TXT files processed with automatic text extraction and cleaning.
- RESTful API with JWT authentication and role-based access control.
- Responsive web interface with history, statistics, and export.
- PostgreSQL persistence with Alembic-managed migrations.

9. Recommendations

1. **Add ROUGE evaluation.** Integrating the ROUGE-N metric against reference summaries would enable quantitative comparison of the four algorithms beyond processing time.
2. **Scanned PDF support.** Adding Tesseract OCR (via `pytesseract`) would handle image-based PDFs, which currently produce no output.
3. **Abstractive comparison mode.** A lightweight abstractive model (e.g., BART or T5 via the Hugging Face pipeline) could be offered as a fifth algorithm with an explicit disclaimer about potential hallucinations.
4. **Batch processing.** Allowing users to upload multiple files in a single request and queue them asynchronously using Celery + Redis would improve usability for large workloads.
5. **Sentence highlighting.** Returning sentence-level relevance scores alongside the summary would allow the frontend to highlight contributing sentences in the original text, improving interpretability.
6. **Containerization.** A Docker Compose configuration would eliminate environment setup friction and make the project easier to evaluate or deploy.
7. **Language support.** NLTK supports several non-English tokenizers; exposing a language selector would make the system useful to a broader audience.

9. References

- [1] Luhn, H. P. (1958). The automatic creation of literature abstracts. *IBM Journal of Research and Development*, 2(2), 159–165.
- [2] Erkan, G., & Radev, D. R. (2004). LexRank: Graph-based lexical centrality as salience in text summarization. *Journal of Artificial Intelligence Research*, 22, 457–479.
- [3] Gong, Y., & Liu, X. (2001). Generic text summarization using relevance measure and latent semantic analysis. In *Proceedings of ACM SIGIR*, pp. 19–25.
- [4] Mihalcea, R., & Tarau, P. (2004). TextRank: Bringing order into texts. In *Proceedings of EMNLP*, pp. 404–411.
- [5] Sparck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21.
- [6] Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media. Available: <https://www.nltk.org/book/>
- [7] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [8] Tichy, M., et al. *Sumy: Automatic text summarizer* (v0.11). <https://github.com/miso-belica/sumy>
- [9] Ramachandran, R., & Jensen, J. J. (2023). FastAPI documentation. <https://fastapi.tiangolo.com>
- [10] Kluyver, T., et al. (2016). Jupyter Notebooks: a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing*, pp. 87–90.